

EN3160 Assignment 1: Intensity Transformations and Neighborhood Filtering

Name: Wijesinghe W. A. P. W.

Index Number: 230727P

GitHub Repository: <https://github.com/Pamodya-Wijesinghe/EN3160-Assignment-1-on-Intensity-Transformations-and-Neighborhood-Filtering>

Q1: Piecewise Linear Intensity Transformation

Applied a piecewise linear transform using given breakpoints to boost contrast. A 256 entry lookup table was built by linear interpolation between the breakpoints and applied to the image using cv2.LUT. the image contrast.

```
In [110... from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
In [111... import numpy as np
import cv2
import matplotlib.pyplot as plt

plt.rcParams['figure.dpi'] = 80

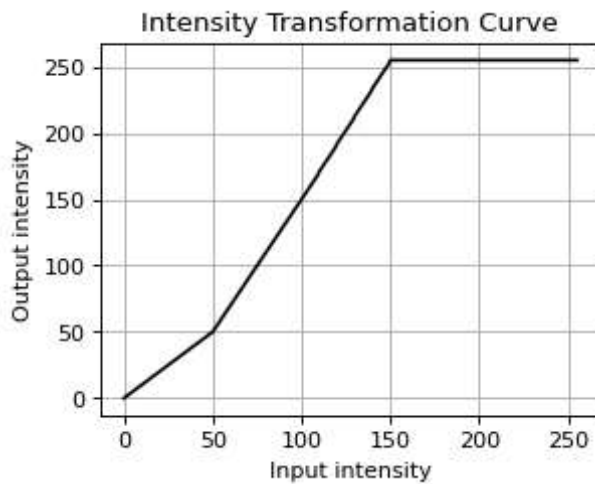
def intensity_transform(im, breakpoints):
    breakpoints = np.array(breakpoints, dtype=np.float64)
    xs = breakpoints[:, 0]
    ys = breakpoints[:, 1]
    x_full = np.arange(256)
    lut = np.interp(x_full, xs, ys).astype(np.uint8)
    transformed = cv2.LUT(im, lut)
    return transformed, lut

im = cv2.imread('/content/drive/MyDrive/EN3160/data/emma.jpeg', cv2.IMREAD_GRAYSCALE)

breakpoints = [[0, 0], [50, 50], [100, 150], [150, 255], [255, 255]]

transformed, lut = intensity_transform(im, breakpoints)
```

```
In [112... plt.figure(figsize=(4,3))
plt.plot(np.arange(256), lut, 'k')
plt.xlabel('Input intensity')
plt.ylabel('Output intensity')
plt.title('Intensity Transformation Curve')
plt.grid(True)
plt.show()
```



```
In [113... fig, axes = plt.subplots(1, 2, figsize=(7,3))
axes[0].imshow(im, cmap='gray')
axes[0].set_title('Original')
axes[0].axis('off')
axes[1].imshow(transformed, cmap='gray')
axes[1].set_title('Transformed')
axes[1].axis('off')
plt.tight_layout()
plt.show()
```



Result: mid tones became brighter and higher contrast, while very dark and very bright areas stayed almost the same, matching the steep part of the curve between 50 and 150. A smoother curve, (0,0), (64,40), (128,128), (192,216), (255,255), gave a softer boost, but the original breakpoints were kept since they made the face clearer without looking harsh.

Q2: Accentuating White Matter and Gray Matter

Used the same intensity_transform function from Q1 with different breakpoints, chosen by checking the brain image histogram. Since the scan had one broad tissue hump (165-240) next to a background peak, white matter was targeted by stretching the upper part (200-240), and gray matter by stretching the lower part (165-195).

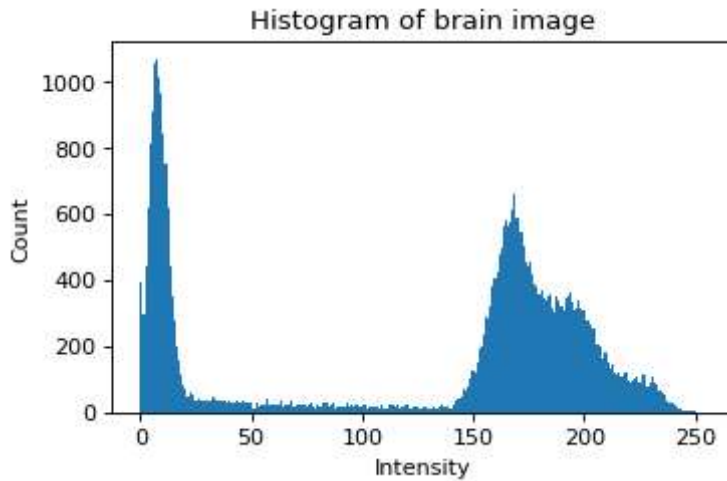
```
In [114... brain = cv2.imread('/content/drive/MyDrive/EN3160/data/brain.jpeg', cv2.IMREAD_G
```

```

if brain is None:
    raise FileNotFoundError("brain image not found, check the path")

plt.figure(figsize=(5,3))
plt.hist(brain.ravel(), bins=256, range=(0,255))
plt.title('Histogram of brain image')
plt.xlabel('Intensity')
plt.ylabel('Count')
plt.show()

```



In [115...

```

white_matter_breakpoints = [[0, 0],[150, 20],[165, 40],[200, 180],
                             [240, 255],[255, 255]]

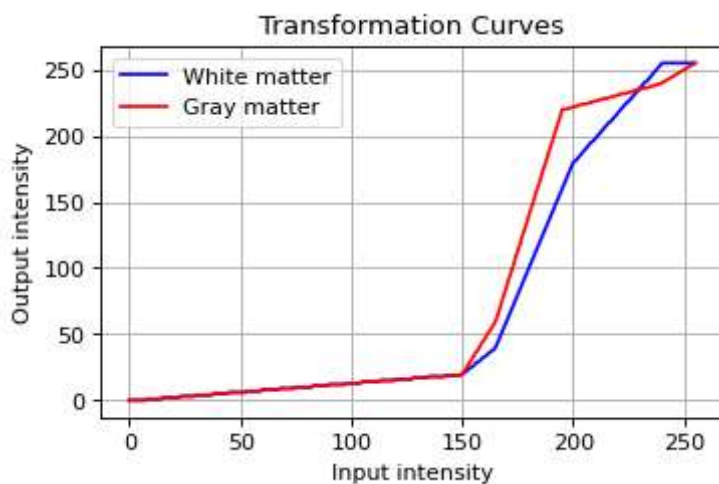
white_result, white_lut = intensity_transform(brain, white_matter_breakpoints)

gray_matter_breakpoints = [[0, 0],[150, 20],[165, 60],[195, 220],
                             [240, 240],[255, 255]]

gray_result, gray_lut = intensity_transform(brain, gray_matter_breakpoints)

plt.figure(figsize=(5,3))
plt.plot(np.arange(256), white_lut, 'b', label='White matter')
plt.plot(np.arange(256), gray_lut, 'r', label='Gray matter')
plt.title('Transformation Curves')
plt.xlabel('Input intensity')
plt.ylabel('Output intensity')
plt.legend()
plt.grid(True)
plt.show()

```

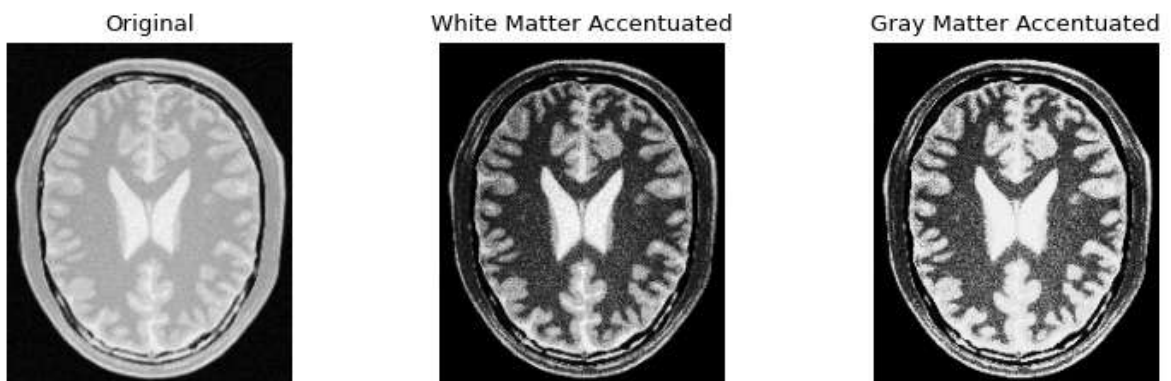


```
In [116... fig, axes = plt.subplots(1, 3, figsize=(10,3))
axes[0].imshow(brain, cmap='gray')
axes[0].set_title('Original')
axes[0].axis('off')

axes[1].imshow(white_result, cmap='gray')
axes[1].set_title('White Matter Accentuated')
axes[1].axis('off')

axes[2].imshow(gray_result, cmap='gray')
axes[2].set_title('Gray Matter Accentuated')
axes[2].axis('off')

plt.tight_layout()
plt.show()
```



Result: the white matter version made outer brain regions clearer, and the gray matter version made inner brain regions clearer, each at the cost of compressing and flattening the other tissue range.

Q3: Gamma Correction on L Plane

Converted the image to Lab color space and applied gamma correction only to the L channel using $\text{output} = 255 * (\text{input}/255)^\gamma$, leaving a and b untouched so colors stay the same. Used $\gamma = 0.6$ (less than 1) to brighten shadow detail in the dark rocks without overexposing the already bright white skirt.

```
In [117... img_bgr = cv2.imread('/content/drive/MyDrive/EN3160/data/rocks.jpeg')

if img_bgr is None:
    raise FileNotFoundError("image not found, check the path")

img_lab = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2LAB)
L, a, b = cv2.split(img_lab)
```

```
In [118... gamma = 0.6 # chosen to brighten shadow detail, adjust based on result

L_norm = L / 255.0
L_gamma = np.power(L_norm, gamma)
L_corrected = np.uint8(L_gamma * 255)

img_lab_corrected = cv2.merge([L_corrected, a, b])
img_corrected_bgr = cv2.cvtColor(img_lab_corrected, cv2.COLOR_LAB2BGR)
```

```
In [119... fig, axes = plt.subplots(2, 2, figsize=(8,5))

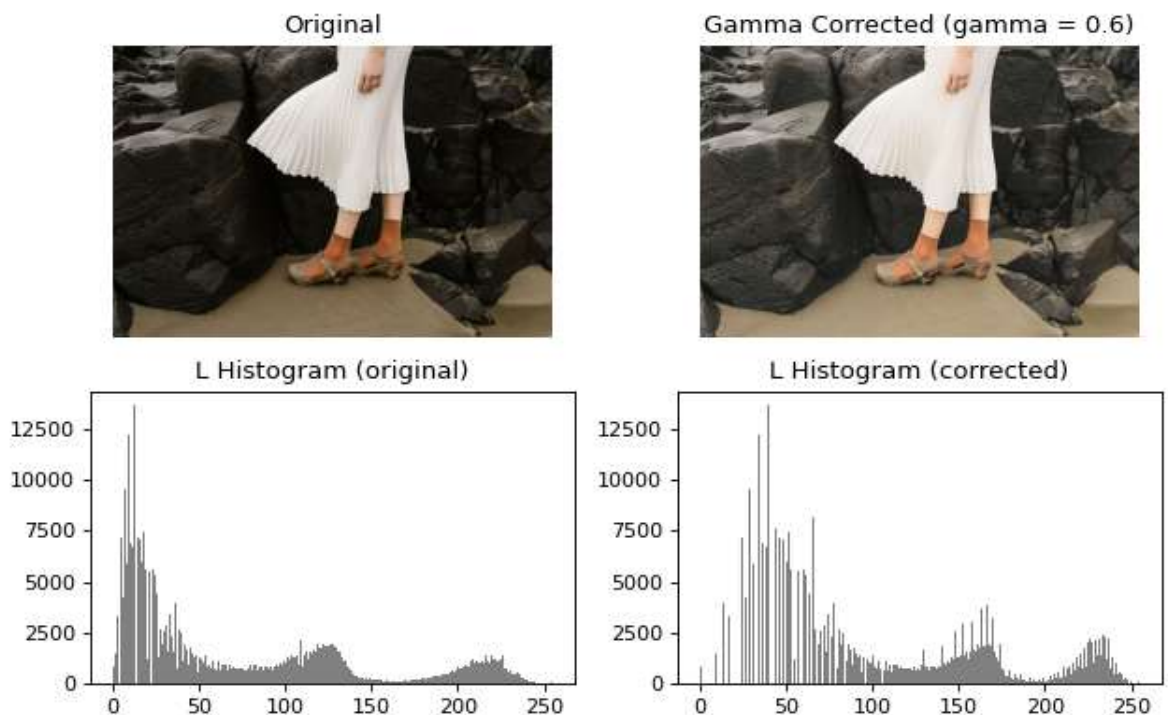
axes[0,0].imshow(cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB))
axes[0,0].set_title('Original')
axes[0,0].axis('off')

axes[0,1].imshow(cv2.cvtColor(img_corrected_bgr, cv2.COLOR_BGR2RGB))
axes[0,1].set_title(f'Gamma Corrected (gamma = {gamma})')
axes[0,1].axis('off')

axes[1,0].hist(L.ravel(), bins=256, range=(0,255), color='gray')
axes[1,0].set_title('L Histogram (original)')

axes[1,1].hist(L_corrected.ravel(), bins=256, range=(0,255), color='gray')
axes[1,1].set_title('L Histogram (corrected)')

plt.tight_layout()
plt.show()
```



Result: the L histogram shifted from bunched up dark values toward the middle and brighter range, and the rocks showed more visible texture, while the skirt stayed close to its original brightness since it was already near 255.

Q4: Vibrance Enhancement

Converted the image to HSV and applied a Gaussian shaped boost to only the saturation plane: $f(x) = \min(x + a128\exp(-(x-128)^2/(2*\sigma^2)), 255)$, with $\sigma = 70$. This boosts mid range saturation the most while leaving very dull or very saturated colors mostly unchanged, giving a more natural vibrance effect than a flat saturation increase. Planes were merged back and converted to BGR.

```
In [120... img_bgr = cv2.imread('/content/drive/MyDrive/EN3160/data/spiderman.jpeg')

if img_bgr is None:
    raise FileNotFoundError("image not found, check the path")

img_hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)
h, s, v = cv2.split(img_hsv)
```

```
In [121... def vibrance_transform(x, a, sigma=70):
    x = x.astype(np.float64)
    boost = a * 128 * np.exp(-((x - 128) ** 2) / (2 * sigma ** 2))
    result = np.minimum(x + boost, 255)
    return result.astype(np.uint8)

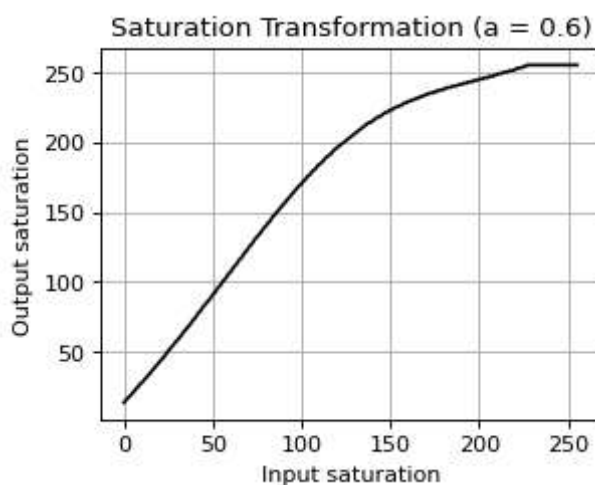
a = 0.6 # adjusted this to control strength between 0 and 1

s_enhanced = vibrance_transform(s, a)
```

```
In [122... img_hsv_enhanced = cv2.merge([h, s_enhanced, v])
img_enhanced_bgr = cv2.cvtColor(img_hsv_enhanced, cv2.COLOR_HSV2BGR)
```

```
In [123... x_vals = np.arange(256)
y_vals = vibrance_transform(x_vals, a)

plt.figure(figsize=(4,3))
plt.plot(x_vals, y_vals, 'k')
plt.title(f'Saturation Transformation (a = {a})')
plt.xlabel('Input saturation')
plt.ylabel('Output saturation')
plt.grid(True)
plt.show()
```



```
In [124... fig, axes = plt.subplots(1, 2, figsize=(8,3))
axes[0].imshow(cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB))
axes[0].set_title('Original')
axes[0].axis('off')
```

```

axes[1].imshow(cv2.cvtColor(img_enhanced_bgr, cv2.COLOR_BGR2RGB))
axes[1].set_title(f'Vibrance Enhanced (a = {a})')
axes[1].axis('off')

plt.tight_layout()
plt.show()

```



Result: $a = 0.6$ gave the best balance, noticeably more vivid colors without looking artificial. $a = 0.9$ oversaturated the red suit, $a = 0.3$ was barely noticeable.

Q5: Histogram Equalization

Wrote a custom histogram equalization function: computed the histogram, took its cumulative sum (CDF), normalized the CDF to 0-255, and used it as a lookup table applied to the image.

```

In [125... def my_histogram_equalization(im):
    # Step 1: compute histogram
    hist, bins = np.histogram(im.ravel(), bins=256, range=(0,256))

    # Step 2: compute cumulative sum
    cdf = np.cumsum(hist)

    # Step 3: normalize cdf to range 0-255
    cdf_normalized = (cdf - cdf.min()) * 255 / (cdf.max() - cdf.min())
    cdf_normalized = cdf_normalized.astype(np.uint8)

    # Step 4: apply as lookup table
    equalized = cdf_normalized[im]

    return equalized, cdf_normalized

```

```

In [126... leaves = cv2.imread('/content/drive/MyDrive/EN3160/data/leaves.tif', cv2.IMREAD_

if leaves is None:
    raise FileNotFoundError("image not found, check the path")

equalized, cdf_normalized = my_histogram_equalization(leaves)

```

```

In [127... fig, axes = plt.subplots(2, 2, figsize=(8,4))

axes[0,0].imshow(leaves, cmap='gray')
axes[0,0].set_title('Original')

```

```

axes[0,0].axis('off')

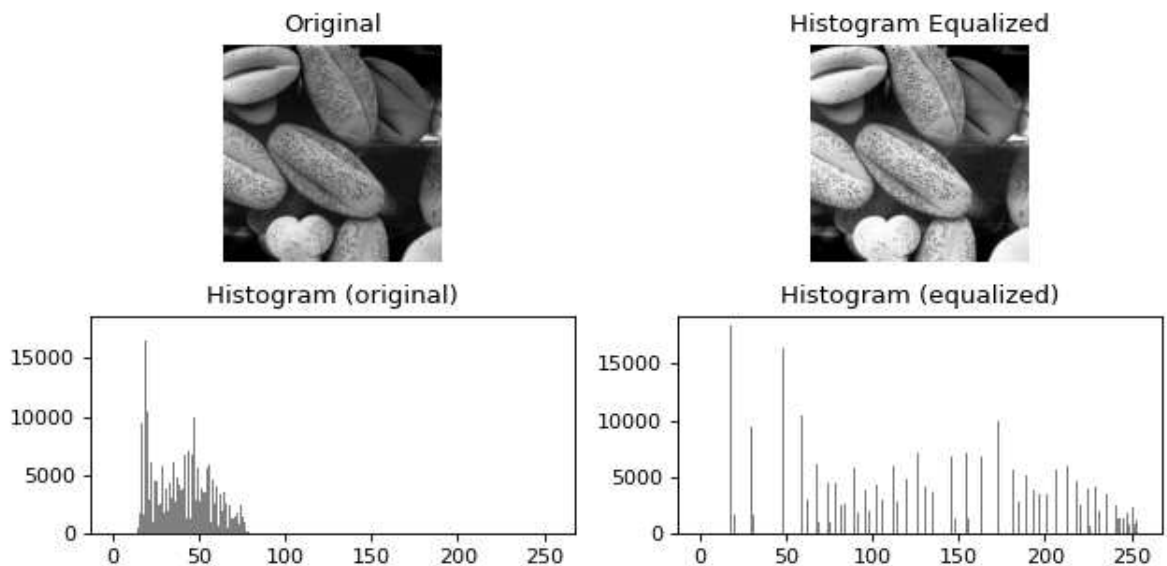
axes[0,1].imshow(equalized, cmap='gray')
axes[0,1].set_title('Histogram Equalized')
axes[0,1].axis('off')

axes[1,0].hist(leaves.ravel(), bins=256, range=(0,255), color='gray')
axes[1,0].set_title('Histogram (original)')

axes[1,1].hist(equalized.ravel(), bins=256, range=(0,255), color='gray')
axes[1,1].set_title('Histogram (equalized)')

plt.tight_layout()
plt.show()

```



In [128...

```

cv_equalized = cv2.equalizeHist(leaves)

diff = np.abs(equalized.astype(int) - cv_equalized.astype(int))
print("Max difference from OpenCV result:", diff.max())
print("Mean difference from OpenCV result:", diff.mean())

```

Max difference from OpenCV result: 18

Mean difference from OpenCV result: 8.976284

Result: the original histogram was bunched in a narrow range (flat, low contrast image), and after equalization it spread across nearly the full 0-255 range, showing much more visible texture. Compared against cv2.equalizeHist, max difference was 18 and mean difference was about 9, a moderate gap likely from small differences in how OpenCV handles CDF normalization, though both results correctly increased contrast in a similar way.

Q6: Histogram Equalization of Foreground Only

Split the image into HSV and thresholded the saturation plane (threshold = 12) to build a binary foreground mask, since the person had noticeably higher saturation than the plain gray background. Used cv2.bitwise_and to extract the foreground, applied the Q5 histogram equalization steps using only foreground pixel statistics, then combined the equalized foreground with the untouched background using the inverted mask.

```
In [129... img_bgr = cv2.imread('/content/drive/MyDrive/EN3160/data/portrait.jpeg')

if img_bgr is None:
    raise FileNotFoundError("image not found, check the path")

img_gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
img_hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)
h, s, v = cv2.split(img_hsv)
```

```
In [130... fig, axes = plt.subplots(1, 3, figsize=(8,3))
axes[0].imshow(h, cmap='gray')
axes[0].set_title('Hue plane')
axes[0].axis('off')

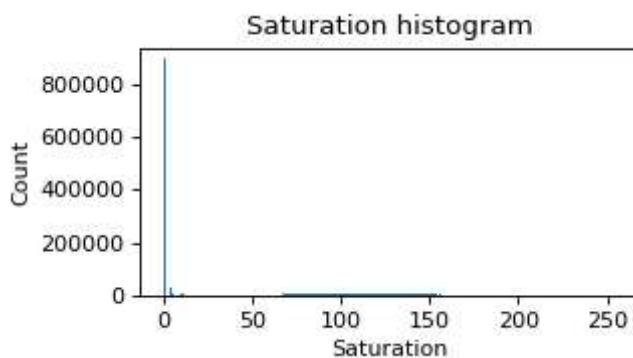
axes[1].imshow(s, cmap='gray')
axes[1].set_title('Saturation plane')
axes[1].axis('off')

axes[2].imshow(v, cmap='gray')
axes[2].set_title('Value plane')
axes[2].axis('off')

plt.tight_layout()
plt.show()
```



```
In [131... # check the saturation histogram first to help choose a threshold
plt.figure(figsize=(4,2))
plt.hist(s.ravel(), bins=256, range=(0,255))
plt.title('Saturation histogram')
plt.xlabel('Saturation')
plt.ylabel('Count')
plt.show()
```



```
In [132... threshold_value = 12

_, mask = cv2.threshold(s, threshold_value, 255, cv2.THRESH_BINARY)
```

```
foreground = cv2.bitwise_and(img_gray, img_gray, mask=mask)
```

```
In [133... # only count pixels where mask is white, ignore the zero background pixels
foreground_pixels = img_gray[mask == 255]

hist, bins = np.histogram(foreground_pixels, bins=256, range=(0,256))
cdf = np.cumsum(hist)
cdf_normalized = (cdf - cdf.min()) * 255 / (cdf.max() - cdf.min())
cdf_normalized = cdf_normalized.astype(np.uint8)

foreground_equalized = np.zeros_like(img_gray)
foreground_equalized[mask == 255] = cdf_normalized[img_gray[mask == 255]]
```

```
In [134... mask_inv = cv2.bitwise_not(mask)
background = cv2.bitwise_and(img_gray, img_gray, mask=mask_inv)

final_result = cv2.add(background, foreground_equalized)
```

```
In [135... fig, axes = plt.subplots(1, 3, figsize=(8,3))

axes[0].imshow(mask, cmap='gray')
axes[0].set_title('Mask')
axes[0].axis('off')

axes[1].imshow(cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB))
axes[1].set_title('Original')
axes[1].axis('off')

axes[2].imshow(final_result, cmap='gray')
axes[2].set_title('Foreground Equalized Result')
axes[2].axis('off')

plt.tight_layout()
plt.show()
```



Result: the mask cleanly covered the person's hair, skin and clothing. The final image shows more contrast and detail in the person, especially the hair, while the background stayed exactly unchanged, confirming equalization was applied only to the intended region.

Q7: Sobel Filtering

Computed the image gradient three ways: (a) cv2.filter2D with the 3x3 Sobel kernels, (b) manual sliding window convolution with loops, (c) separable 1D convolutions using the outer product decomposition of the Sobel kernel.

```
In [136... einstein = cv2.imread('/content/drive/MyDrive/EN3160/data/einstein.jpeg', cv2.IM
if einstein is None:
    raise FileNotFoundError("image not found, check the path")

einstein_f = einstein.astype(np.float64)
```

```
In [137... sobel_x_kernel = np.array([[1, 0, -1],[2, 0, -2],[1, 0, -1]], dtype=np.float64)

sobel_y_kernel = np.array([[1, 2, 1],[0, 0, 0],[-1, -2, -1]], dtype=np.float64)

grad_x = cv2.filter2D(einstein_f, -1, sobel_x_kernel, borderType=cv2.BORDER_CONS
grad_y = cv2.filter2D(einstein_f, -1, sobel_y_kernel, borderType=cv2.BORDER_CONS

gradient_magnitude_a = np.sqrt(grad_x**2 + grad_y**2)
gradient_magnitude_a = np.uint8(255 * gradient_magnitude_a / gradient_magnitude_
```

```
In [138... def manual_convolution(im, kernel):
    kh, kw = kernel.shape
    pad_h, pad_w = kh // 2, kw // 2

    padded = np.pad(im, ((pad_h, pad_h), (pad_w, pad_w)), mode='constant')
    output = np.zeros_like(im, dtype=np.float64)

    rows, cols = im.shape
    for i in range(rows):
        for j in range(cols):
            region = padded[i:i+kh, j:j+kw]
            output[i, j] = np.sum(region * kernel)

    return output

grad_x_manual = manual_convolution(einstein_f, sobel_x_kernel)
grad_y_manual = manual_convolution(einstein_f, sobel_y_kernel)

gradient_magnitude_b = np.sqrt(grad_x_manual**2 + grad_y_manual**2)
gradient_magnitude_b = np.uint8(255 * gradient_magnitude_b / gradient_magnitude_
```

```
In [139... # sobel_x kernel splits into a column vector and a row vector
col_kernel = np.array([[1], [2], [1]], dtype=np.float64)
row_kernel = np.array([[1, 0, -1]], dtype=np.float64)

grad_x_sep = cv2.filter2D(einstein_f, -1, col_kernel, borderType=cv2.BORDER_CONS
grad_x_sep = cv2.filter2D(grad_x_sep, -1, row_kernel, borderType=cv2.BORDER_CONS

# same idea for the vertical kernel, just transpose the two 1D kernels
col_kernel_y = np.array([[1], [0], [-1]], dtype=np.float64)
row_kernel_y = np.array([[1, 2, 1]], dtype=np.float64)

grad_y_sep = cv2.filter2D(einstein_f, -1, col_kernel_y, borderType=cv2.BORDER_CC
grad_y_sep = cv2.filter2D(grad_y_sep, -1, row_kernel_y, borderType=cv2.BORDER_CC

gradient_magnitude_c = np.sqrt(grad_x_sep**2 + grad_y_sep**2)
gradient_magnitude_c = np.uint8(255 * gradient_magnitude_c / gradient_magnitude_
```

```
In [140... fig, axes = plt.subplots(1, 4, figsize=(8,3))

axes[0].imshow(einstein, cmap='gray')
```

```

axes[0].set_title('Original')
axes[0].axis('off')

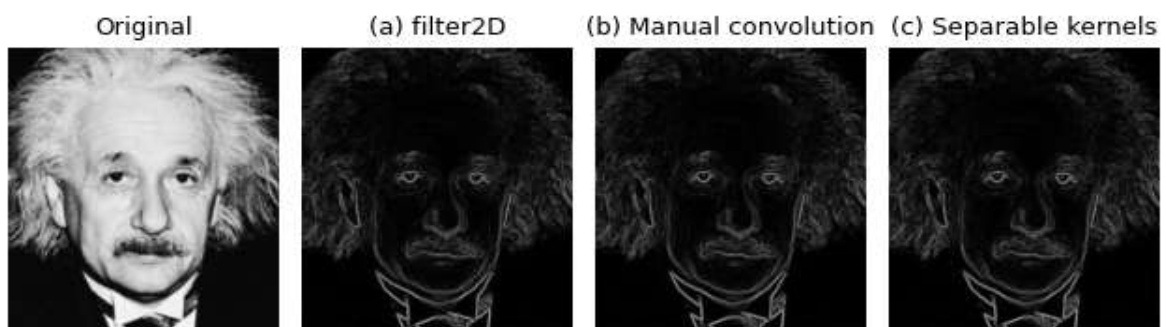
axes[1].imshow(gradient_magnitude_a, cmap='gray')
axes[1].set_title('(a) filter2D')
axes[1].axis('off')

axes[2].imshow(gradient_magnitude_b, cmap='gray')
axes[2].set_title('(b) Manual convolution')
axes[2].axis('off')

axes[3].imshow(gradient_magnitude_c, cmap='gray')
axes[3].set_title('(c) Separable kernels')
axes[3].axis('off')

plt.tight_layout()
plt.show()

```



```

In [141... diff_ab = np.abs(gradient_magnitude_a.astype(int) - gradient_magnitude_b.astype(int))
diff_ac = np.abs(gradient_magnitude_a.astype(int) - gradient_magnitude_c.astype(int))

print("Max difference between (a) and (b):", diff_ab.max())
print("Max difference between (a) and (c):", diff_ac.max())

```

Max difference between (a) and (b): 0
 Max difference between (a) and (c): 0

Result: all three gave identical edge maps (max difference 0) once border handling was matched, since filter2D defaults to reflected padding while the manual version used zero padding, causing a large mismatch until `borderType=cv2.BORDER_CONSTANT` was set for both filter2D calls. Manual convolution was noticeably slower due to plain Python loops; separable filtering was the most efficient while giving the same result.

Q8: Image Zooming

Wrote a custom zoom function supporting nearest neighbor (rounds to closest pixel) and bilinear interpolation (blends the 4 nearest pixels using fractional distance weights). Tested by zooming two small images up by factor 4 and comparing against the true large originals using normalized SSD.

```

In [142... def zoom_nearest_neighbor(im, s):
    h, w = im.shape[:2]
    new_h, new_w = int(round(h * s)), int(round(w * s))

    if im.ndim == 2:

```

```

        output = np.zeros((new_h, new_w), dtype=im.dtype)
    else:
        output = np.zeros((new_h, new_w, im.shape[2]), dtype=im.dtype)

    for i in range(new_h):
        for j in range(new_w):
            orig_i = min(int(round(i / s)), h - 1)
            orig_j = min(int(round(j / s)), w - 1)
            output[i, j] = im[orig_i, orig_j]

    return output

```

In [143...

```

def zoom_bilinear(im, s):
    h, w = im.shape[:2]
    new_h, new_w = int(round(h * s)), int(round(w * s))

    im_f = im.astype(np.float64)

    if im.ndim == 2:
        output = np.zeros((new_h, new_w), dtype=np.float64)
    else:
        output = np.zeros((new_h, new_w, im.shape[2]), dtype=np.float64)

    for i in range(new_h):
        for j in range(new_w):
            orig_i = i / s
            orig_j = j / s

            i0 = int(np.floor(orig_i))
            j0 = int(np.floor(orig_j))
            i1 = min(i0 + 1, h - 1)
            j1 = min(j0 + 1, w - 1)
            i0 = min(i0, h - 1)
            j0 = min(j0, w - 1)

            di = orig_i - i0
            dj = orig_j - j0

            top = im_f[i0, j0] * (1 - dj) + im_f[i0, j1] * dj
            bottom = im_f[i1, j0] * (1 - dj) + im_f[i1, j1] * dj
            output[i, j] = top * (1 - di) + bottom * di

    return np.uint8(np.clip(output, 0, 255))

```

In [144...

```

small_img1 = cv2.imread('/content/drive/MyDrive/EN3160/data/small3.png')
large_img1 = cv2.imread('/content/drive/MyDrive/EN3160/data/large3.png')

small_img2 = cv2.imread('/content/drive/MyDrive/EN3160/data/small2.png')
large_img2 = cv2.imread('/content/drive/MyDrive/EN3160/data/large2.png')

if small_img1 is None or large_img1 is None or small_img2 is None or large_img2 is None:
    raise FileNotFoundError("one or more images not found, check the paths")

```

In [145...

```

zoomed_nn_1 = zoom_nearest_neighbor(small_img1, 4)
zoomed_bilinear_1 = zoom_bilinear(small_img1, 4)

zoomed_nn_2 = zoom_nearest_neighbor(small_img2, 4)
zoomed_bilinear_2 = zoom_bilinear(small_img2, 4)

```

```
In [146... def normalized_ssd(im1, im2):
    # make sure shapes match exactly before comparing
    h = min(im1.shape[0], im2.shape[0])
    w = min(im1.shape[1], im2.shape[1])
    im1_c = im1[:h, :w].astype(np.float64)
    im2_c = im2[:h, :w].astype(np.float64)

    ssd = np.sum((im1_c - im2_c) ** 2)
    normalized = ssd / (h * w * 255**2)
    return normalized

ssd_nn_1 = normalized_ssd(zoomed_nn_1, large_img1)
ssd_bilinear_1 = normalized_ssd(zoomed_bilinear_1, large_img1)

ssd_nn_2 = normalized_ssd(zoomed_nn_2, large_img2)
ssd_bilinear_2 = normalized_ssd(zoomed_bilinear_2, large_img2)

print("Image 1 - Nearest neighbor SSD:", ssd_nn_1)
print("Image 1 - Bilinear SSD:", ssd_bilinear_1)
print("Image 2 - Nearest neighbor SSD:", ssd_nn_2)
print("Image 2 - Bilinear SSD:", ssd_bilinear_2)
```

```
Image 1 - Nearest neighbor SSD: 0.0029817475583643043
Image 1 - Bilinear SSD: 0.002258854233414499
Image 2 - Nearest neighbor SSD: 0.005984501286257715
Image 2 - Bilinear SSD: 0.0045242134204898435
```

```
In [147... fig, axes = plt.subplots(2, 3, figsize=(8,4))

axes[0,0].imshow(cv2.cvtColor(small_img1, cv2.COLOR_BGR2RGB))
axes[0,0].set_title('Small Image 1')
axes[0,0].axis('off')

axes[0,1].imshow(cv2.cvtColor(zoomed_nn_1, cv2.COLOR_BGR2RGB))
axes[0,1].set_title('Nearest Neighbor x4')
axes[0,1].axis('off')

axes[0,2].imshow(cv2.cvtColor(zoomed_bilinear_1, cv2.COLOR_BGR2RGB))
axes[0,2].set_title('Bilinear x4')
axes[0,2].axis('off')

axes[1,0].imshow(cv2.cvtColor(small_img2, cv2.COLOR_BGR2RGB))
axes[1,0].set_title('Small Image 2')
axes[1,0].axis('off')

axes[1,1].imshow(cv2.cvtColor(zoomed_nn_2, cv2.COLOR_BGR2RGB))
axes[1,1].set_title('Nearest Neighbor x4')
axes[1,1].axis('off')

axes[1,2].imshow(cv2.cvtColor(zoomed_bilinear_2, cv2.COLOR_BGR2RGB))
axes[1,2].set_title('Bilinear x4')
axes[1,2].axis('off')

plt.tight_layout()
plt.show()
```



Result: bilinear gave lower SSD than nearest neighbor for both images (Image 1: 0.00298 vs 0.00226; Image 2: 0.00598 vs 0.00452), confirming bilinear is more accurate since it handles gradual intensity changes, while nearest neighbor can only copy existing pixel values, causing blocky results. Image 2 had higher SSD than Image 1 for both methods, likely due to finer texture that is harder to recover after downsampling and zooming back up.

Q9: Foreground Segmentation and Background Blur

Used grabCut with a rectangle around the flower to build a foreground mask, converting the 4 grabCut labels into a simple binary mask. Blurred the background heavily with a Gaussian filter and combined it with the sharp foreground using the mask, creating a depth of field effect.

```
In [148... flower_bgr = cv2.imread('/content/drive/MyDrive/EN3160/data/flower.jpeg')

if flower_bgr is None:
    raise FileNotFoundError("image not found, check the path")

flower_rgb = cv2.cvtColor(flower_bgr, cv2.COLOR_BGR2RGB)

In [149... mask = np.zeros(flower_bgr.shape[:2], np.uint8)

bgd_model = np.zeros((1, 65), np.float64)
fgd_model = np.zeros((1, 65), np.float64)

# rectangle format: (x, y, width, height)
rect = (50, 130, 570, 700) # adjust to actually include stem and bud

cv2.grabCut(flower_bgr, mask, rect, bgd_model, fgd_model, 5, cv2.GC_INIT_WITH_RE

binary_mask = np.where((mask == 2) | (mask == 0), 0, 1).astype('uint8')

In [150... foreground = flower_bgr * binary_mask[:, :, np.newaxis]
background = flower_bgr * (1 - binary_mask[:, :, np.newaxis])
```

```
In [151... blurred_background = cv2.GaussianBlur(flower_bgr, (35, 35), 0)

mask_3ch = np.repeat(binary_mask[:, :, np.newaxis], 3, axis=2)

enhanced = np.where(mask_3ch == 1, flower_bgr, blurred_background)
```

```
In [152... fig, axes = plt.subplots(1, 5, figsize=(8,3))

axes[0].imshow(binary_mask, cmap='gray')
axes[0].set_title('Mask')
axes[0].axis('off')

axes[1].imshow(cv2.cvtColor(foreground, cv2.COLOR_BGR2RGB))
axes[1].set_title('Foreground')
axes[1].axis('off')

axes[2].imshow(cv2.cvtColor(background, cv2.COLOR_BGR2RGB))
axes[2].set_title('Background')
axes[2].axis('off')

axes[3].imshow(flower_rgb)
axes[3].set_title('Original')
axes[3].axis('off')

axes[4].imshow(cv2.cvtColor(enhanced, cv2.COLOR_BGR2RGB))
axes[4].set_title('Blurred Background')
axes[4].axis('off')

plt.tight_layout()
plt.show()
```



Result: an initial tight rectangle left the stem and bud outside the mask; widening it fixed this. The area just beyond the flower's edge appears dark because the hard binary mask has no blending at the boundary. Real edge pixels are naturally a mix of flower and background color due to camera anti-aliasing, and blurring pulls these mixed pixels toward the darker green leaf color, creating a visible dark ring right next to the sharp foreground cutoff.

Q10: Bilateral Filtering

A bilateral filter smooths using two weights per neighbor: a spatial weight (σ_s , based on distance) and a range weight (σ_r , based on intensity difference), so edges with large intensity jumps stay sharp while flat areas still get smoothed. Used

cv2.bilateralFilter, then wrote a custom version with nested loops computing both weights directly, compared against OpenCV's version and a plain Gaussian blur.

```
In [153... lake = cv2.imread('/content/drive/MyDrive/EN3160/data/lake.jpeg')

if lake is None:
    raise FileNotFoundError("image not found, check the path")

lake_rgb = cv2.cvtColor(lake, cv2.COLOR_BGR2RGB)
```

```
In [154... sigma_s = 15    # spatial sigma
sigma_r = 75    # range (color) sigma
d = 9          # diameter of pixel neighborhood

bilateral_cv = cv2.bilateralFilter(lake, d, sigma_r, sigma_s)
gaussian_blur = cv2.GaussianBlur(lake, (9,9), sigma_s)

fig, axes = plt.subplots(1, 3, figsize=(8,3))
axes[0].imshow(lake_rgb)
axes[0].set_title('Original')
axes[0].axis('off')

axes[1].imshow(cv2.cvtColor(gaussian_blur, cv2.COLOR_BGR2RGB))
axes[1].set_title('Gaussian Blur')
axes[1].axis('off')

axes[2].imshow(cv2.cvtColor(bilateral_cv, cv2.COLOR_BGR2RGB))
axes[2].set_title('Bilateral Filter (OpenCV)')
axes[2].axis('off')

plt.tight_layout()
plt.show()
```



```
In [155... def my_bilateral_filter(im, d, sigma_s, sigma_r):
    im_f = im.astype(np.float64)
    pad = d // 2
    padded = np.pad(im_f, ((pad,pad),(pad,pad),(0,0)), mode='reflect')

    h, w, c = im_f.shape
    output = np.zeros_like(im_f)

    # precompute spatial gaussian weights, these don't depend on pixel values
    ax = np.arange(-pad, pad+1)
    xx, yy = np.meshgrid(ax, ax)
```

```

spatial_weights = np.exp(-(xx**2 + yy**2) / (2 * sigma_s**2))

for i in range(h):
    for j in range(w):
        region = padded[i:i+d, j:j+d, :]
        center = padded[i+pad, j+pad, :]

        # range weight based on color difference from center pixel
        diff = region - center
        range_weights = np.exp(-np.sum(diff**2, axis=2) / (2 * sigma_r**2))

        weights = spatial_weights * range_weights
        weights_sum = np.sum(weights)

        for ch in range(c):
            output[i, j, ch] = np.sum(region[:, :, ch] * weights) / weights_sum

return np.uint8(np.clip(output, 0, 255))

```

```

In [156... # test on a smaller crop to keep runtime reasonable
lake_small = lake[100:250, 100:250]

bilateral_custom = my_bilateral_filter(lake_small, d, sigma_s, sigma_r)
bilateral_cv_small = cv2.bilateralFilter(lake_small, d, sigma_r, sigma_s)
gaussian_small = cv2.GaussianBlur(lake_small, (9,9), sigma_s)

```

```

In [157... fig, axes = plt.subplots(1, 4, figsize=(8,3))

axes[0].imshow(cv2.cvtColor(lake_small, cv2.COLOR_BGR2RGB))
axes[0].set_title('Original (cropped)')
axes[0].axis('off')

axes[1].imshow(cv2.cvtColor(gaussian_small, cv2.COLOR_BGR2RGB))
axes[1].set_title('Gaussian Blur')
axes[1].axis('off')

axes[2].imshow(cv2.cvtColor(bilateral_cv_small, cv2.COLOR_BGR2RGB))
axes[2].set_title('Bilateral (OpenCV)')
axes[2].axis('off')

axes[3].imshow(cv2.cvtColor(bilateral_custom, cv2.COLOR_BGR2RGB))
axes[3].set_title('Bilateral (Custom)')
axes[3].axis('off')

plt.tight_layout()
plt.show()

```



```

In [158... def normalized_ssd(im1, im2):
    im1_f = im1.astype(np.float64)

```

```
im2_f = im2.astype(np.float64)
ssd = np.sum((im1_f - im2_f) ** 2)
normalized = ssd / (im1.size * 255**2)
return normalized

ssd_custom_vs_cv = normalized_ssd(bilateral_custom, bilateral_cv_small)
ssd_gaussian_vs_cv = normalized_ssd(gaussian_small, bilateral_cv_small)

print("Normalized SSD, custom bilateral vs OpenCV bilateral:", ssd_custom_vs_cv)
print("Normalized SSD, Gaussian blur vs OpenCV bilateral:", ssd_gaussian_vs_cv)
```

Normalized SSD, custom bilateral vs OpenCV bilateral: 0.0006195155709342561
Normalized SSD, Gaussian blur vs OpenCV bilateral: 0.0108240456804363

Result: Gaussian blur smoothed edges as much as flat areas, while both bilateral filters preserved edges like mountain outlines. Normalized SSD: custom vs OpenCV bilateral = 0.00062, Gaussian vs OpenCV bilateral = 0.01082, about 17x higher, confirming the custom implementation closely matches OpenCV's edge preserving behavior while Gaussian blur behaves quite differently. Tested on a cropped patch only, since the nested loop version is slow in plain Python.